

Geo Patroni Cluster Management

Setup replication for a single node

These directions are for setting up archive recovery in geo environments containing a “main” VM running most GitLab services, including postgres. The database here is a singleton and not managed by patroni. Postgres here is installed by the omnibus gitlab-ee package.

Setting up replication is necessary if a node is installed from scratch or if we lost replication for too long and the necessary WAL files are not on the node we are replicating from anymore or if we need to upgrade postgres to a newer major version.

We are using wal-g to retrieve the initial base-backup and WAL files from GCS and then Postgres is automatically switching over to streaming replication (see recovery.conf). As of writing, `geo-secondary-01-sv-gstg.c.gitlab-staging-1.internal` is streaming from `patroni-05-db-gstg.c.gitlab-staging-1.internal`.

- `chef-client-disable <issue-link>`
- `gitlab-ctl stop`
- `gitlab-ctl start geo-postgresql` (needed for the next step)
- `gitlab-rake geo:db:drop`
- make backup of PGDATA conf files (wal-g will not copy them from the streaming replica and recovery.conf is setup manually):
 - `mkdir -p /var/opt/gitlab/postgresql/data.bak; cp -a /var/opt/gitlab/postgresql/data/*.conf /var/opt/gitlab/postgresql/data.bak`
- make sure postgresql.conf matches primary config for `max_connections`
- Find the most recent db backup in GCS:

```
cd /tmp/; sudo -u gitlab-psql envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list
name                               last_modified          wal_segment_backup_start
base_00000009000004B95000000FC 2020-07-07T14:51:36Z 00000009000004B95000000FC
base_00000009000004B96000000DB 2020-07-07T16:43:26Z 00000009000004B96000000DB
```

The first column, “name”, is what you’ll want to copy for later.

Take the newest backup, but make sure that the name really contains the highest segment number (sort should ensure that) - the modification date might not be reliable in case there are GCS life cycle policies involved. The older the backup we take, the more WAL files need to be replayed later.

- `rm -rf /var/opt/gitlab/postgresql/data/*`
- move old data away:

```
rm -rf /var/opt/gitlab/git-data/repositories.old
mv /var/opt/gitlab/git-data/repositories /var/opt/gitlab/git-data/repositories.old
mkdir -p /var/opt/gitlab/git-data/repositories
chown git:git /var/opt/gitlab/git-data/repositories
```

```
rm -rf /var/opt/gitlab/gitlab-rails/shared.old
mv /var/opt/gitlab/gitlab-rails/shared /var/opt/gitlab/gitlab-rails/shared.old
mkdir -p /var/opt/gitlab/gitlab-rails/shared
chown git:gitlab-www /var/opt/gitlab/gitlab-rails/shared
```

```
rm -rf /var/opt/gitlab/gitlab-rails/uploads.old
mv /var/opt/gitlab/gitlab-rails/uploads /var/opt/gitlab/gitlab-rails/uploads.old
mkdir -p /var/opt/gitlab/gitlab-rails/uploads
chown git /var/opt/gitlab/gitlab-rails/uploads
```

- **Only** in case you want to upgrade the major postgres version or gitlab-ee:
 - Re-Install gitlab-ee manually: `dpkg -r gitlab-ee && apt-get -y install gitlab-ee=13.2....`
 - * take the same version as found in staging; run `dpkg -l gitlab-ee` on `web-01-sv-gstg.c.gitlab-staging-1.internal` to find the right version
 - `gitlab-ctl reconfigure`
 - `gitlab-ctl stop`
 - Clean up current PGDATA again: `rm -rf /var/opt/gitlab/postgresql/data/*`
- In a tmux:
 - `cd /tmp; sudo -u gitlab-psql PGHOST=/var/opt/gitlab/postgresql /usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-fetch /var/opt/gitlab/postgresql/data base_00000...`
 - * **take the backup name found above**; this might take 20m or so to finish
- Restore recovery configurations: `cat /var/opt/gitlab/postgresql/data.bak/recovery.conf | grep -v standby_mode >> /var/opt/gitlab/postgresql/data/gitlab-geo.conf`
 - it should look like this:


```
restore_command = '/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-fetch "%f"
recovery_target_timeline = 'latest'
```

Omit this setting if not using streaming replication and relying solely on
archive recovery.
primary_conninfo = 'user=gitlab-replicator password=<password> host=<target> port=5432

Target should be the IP of a replica node in the main postgres cluster, patroni-tagged “nofailover,noloadbalance”. This should have already been set up in advance, and the tags can be examined by the chef attribute `gitlab-patroni.patroni.conf.tags`.

The password can be obtained from the `recovery.conf` file on this replica.

`chown` it to `gitlab-psql:gitlab-psql` and `chmod` it to 600 (if needed).

- Create `standby.signal`:

```
touch /var/opt/gitlab/postgresql/data/standby.signal
chown gitlab-psql:gitlab-psql /var/opt/gitlab/postgresql/data/standby.signal
chmod 0644 /var/opt/gitlab/postgresql/data/standby.signal
```

- `gitlab-ctl reconfigure`

- This will cause postgres to start at some point, and you will see the message “psql: FATAL: the database system is starting up” repeated for some time. This might cause the reconfigure operation to time out. If this happens, periodically try to connect to postgres (`gitlab-psql`) to know when it has completed the crash recovery phase or check `/var/log/gitlab/postgresql/current`, and reconfigure again.

- Read `/var/log/gitlab/postgresql/current` and check for errors.
- `gitlab-ctl start`
- `curl localhost:9187/metrics`. If it hangs, `sv restart postgres_exporter`
- `chef-client-enable`
- Check the replication lag for this node. As long as it is generally decreasing over time, archive recovery is working.
- Once the replication lag is near zero, the geo postgres should switch to using streaming replication rather than archive recovery. You can check this is working by `sudo gitlab-psql -c "select * from pg_stat_wal_receiver;"`. You should see status as 'streaming'.
- Setup foreign tables and log cursor (see handbook)
 - `gitlab-rake geo:db:setup`
 - `gitlab-rake gitlab:geo:check`
 - `gitlab-rake geo:status`
 - `gitlab-ctl hup puma`
 - `gitlab-ctl restart sidekiq`
 - `gitlab-ctl restart geo-logcursor`
- Check <https://staging.gitlab.com/admin/geo/nodes> and <https://geo.staging.gitlab.com/admin/geo/nodes> for status

Setup replication for a patroni-managed cluster

These directions are for setting up archive recovery in geo environments containing a patroni-managed postgres cluster, external to the GitLab service.

Geo patroni cluster is a standby cluster replicating from production via wal archive. If the replication is broken we will have to resetup replication for the entire cluster following below steps:

1. Stop patroni:

```
knife ssh roles:dr-base-db-patroni 'sudo systemctl stop patroni'
knife ssh roles:dr-base-db-patroni 'consul kv delete -recurse service/pg-ha-cluster'
```

2. Backup config files and delete data directory. We need to backup config files because we will use wal-g backups from production to restore the data directory, however wal-g backups does not contain config files. We will have to copy the backed up config files back to data directory after restore:

```
knife ssh roles:dr-base-db-patroni 'sudo cp /var/opt/gitlab/postgresql/data/pg_hba.conf /var/opt/gitlab/postgresql/data/pg_hba.conf.bak'
knife ssh roles:dr-base-db-patroni 'sudo cp /var/opt/gitlab/postgresql/data/pg_ident.conf /var/opt/gitlab/postgresql/data/pg_ident.conf.bak'
knife ssh roles:dr-base-db-patroni 'sudo rm -rf /var/opt/gitlab/postgresql/data' # TAKE CARE!
knife ssh roles:dr-base-db-patroni 'sudo mkdir /var/opt/gitlab/postgresql/data'
knife ssh roles:dr-base-db-patroni 'sudo chown gitlab-psql:gitlab-psql /var/opt/gitlab/postgresql/data'
```

3. Check the latest production backup is available from any node. Make sure the latest wal_segment_backup_start is within 24hrs. If not, report to DBRE because production backup is breaking.

```
/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list | sort
```

```
name last_modified expanded_size_bytes wal_segment_backup_start wal_segment_offset_backup
base_0000001400010476000000023_02614680 2019-06-15 04:27:42.235000+00:00 00000014000104760000000000000000
base_00000014000104FA0000000C1_07405208 2019-06-17 08:21:45.506000+00:00 00000014000104FA0000000000000000
base_00000014000108830000000AB_14587176 2019-06-22 06:52:57.460000+00:00 0000001400010883000000000000000
base_00000014000109D400000008E_00851408 2019-06-25 08:16:19.988000+00:00 00000014000109D4000000000000000
base_0000001400010A8C0000000E8_13030128 2019-06-26 04:45:10.121000+00:00 0000001400010A8C000000000000000
base_0000001400010B4C00000008B_12535584 2019-06-27 07:24:35.529000+00:00 0000001400010B4C000000000000000
```

4. Start patroni in one of the node. This node will likely become the leader node after the restore. Here we choose 01 node as the leader node.

```
ssh patroni-01-db-dr.c.gitlab-dr.internal
sudo su
systemctl start patroni
```

5. Check patroni log to make sure it started patroni and is restoring from production. If for any reason the patroni is not started, try steps in step

1.

```
tail -f /var/log/gitlab/patroni/patroni.log
```

6. While 01 node is restoring, we can start manually restoring 02 and 03 node in parallel to speed up the process. Repeat below steps in both 02 and 03

nodes. The restore process will take several hours, please consider using tmux session to execute the restore command in line 3.

```
ssh patroni-02-db-dr.c.gitlab-dr.internal
sudo su - gitlab-psql
latest="$( \
    /usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list \
    | grep base \
    | sort -nk1 \
    | tail -1 \
    | awk '{print $1}' \
)"
```

```
/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-fetch /var/opt/gitlab/postgresql
```

7. While the restore is in process, copy back the config files we took at step 2 to make sure Postgresql service can start properly after the restore.

```
knife ssh roles:dr-base-db-patroni 'sudo cp /var/opt/gitlab/postgresql/pg_hba.conf.${da}
knife ssh roles:dr-base-db-patroni 'sudo cp /var/opt/gitlab/postgresql/pg_ident.conf.${da}
```

8. Start patroni service on 02 and 03 nodes after the restore is completed.

```
systemctl start patroni
```

9. Now the patroni cluster should be established

```
gitlab-patronictl list
```

Cluster	Member	Host	Role	Status
pg-ha-cluster	patroni-01-db-dr.c.gitlab-dr.internal	10.251.9.101	Leader	running
pg-ha-cluster	patroni-02-db-dr.c.gitlab-dr.internal	10.251.9.102	Replica	running
pg-ha-cluster	patroni-03-db-dr.c.gitlab-dr.internal	10.251.9.103	Replica	running

10. You may notice the non-leader nodes still have very large replication lags. The reason is that all nodes (including the leader) must replay all the wal archive files from the time the backup was taken till the current time. This replay process would take several hours.

During the replay process, the leader node may run out of disk spaces due to inactive replication slots because the replica nodes are not caught up yet.

We can stop and restart patroni services on the **replica** nodes (don't do it on the leader node!) every 2-4 hours to let the leader node clean up spaces itself. You can do it manually, or setup temporary cron jobs to do it like below (don't forget to remove these jobs after the replication is caught up!):

```
crontab -l
0 0/2 * * * /bin/systemctl stop patroni
10 0/2 * * * /bin/systemctl start patroni
```

Related

See patroni management for other patroni related management operations.